

LESSON 1
**ABSTRACTION,
SORTING &
SEARCHING
ALGORITHMS**



**IC217
DATA
STRUCTURES**



Java programmer



01

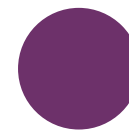
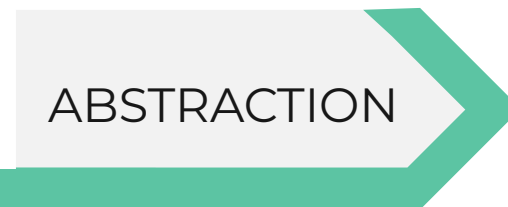
02

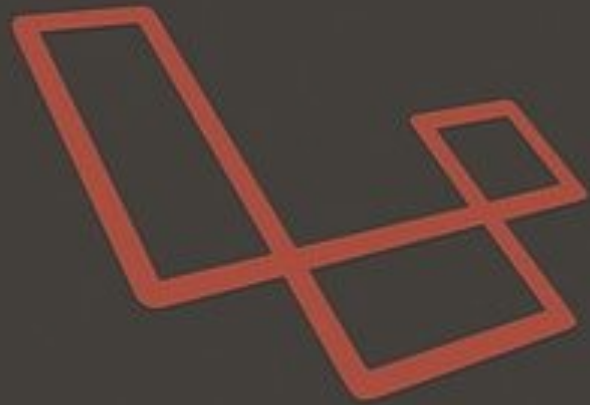
03

04

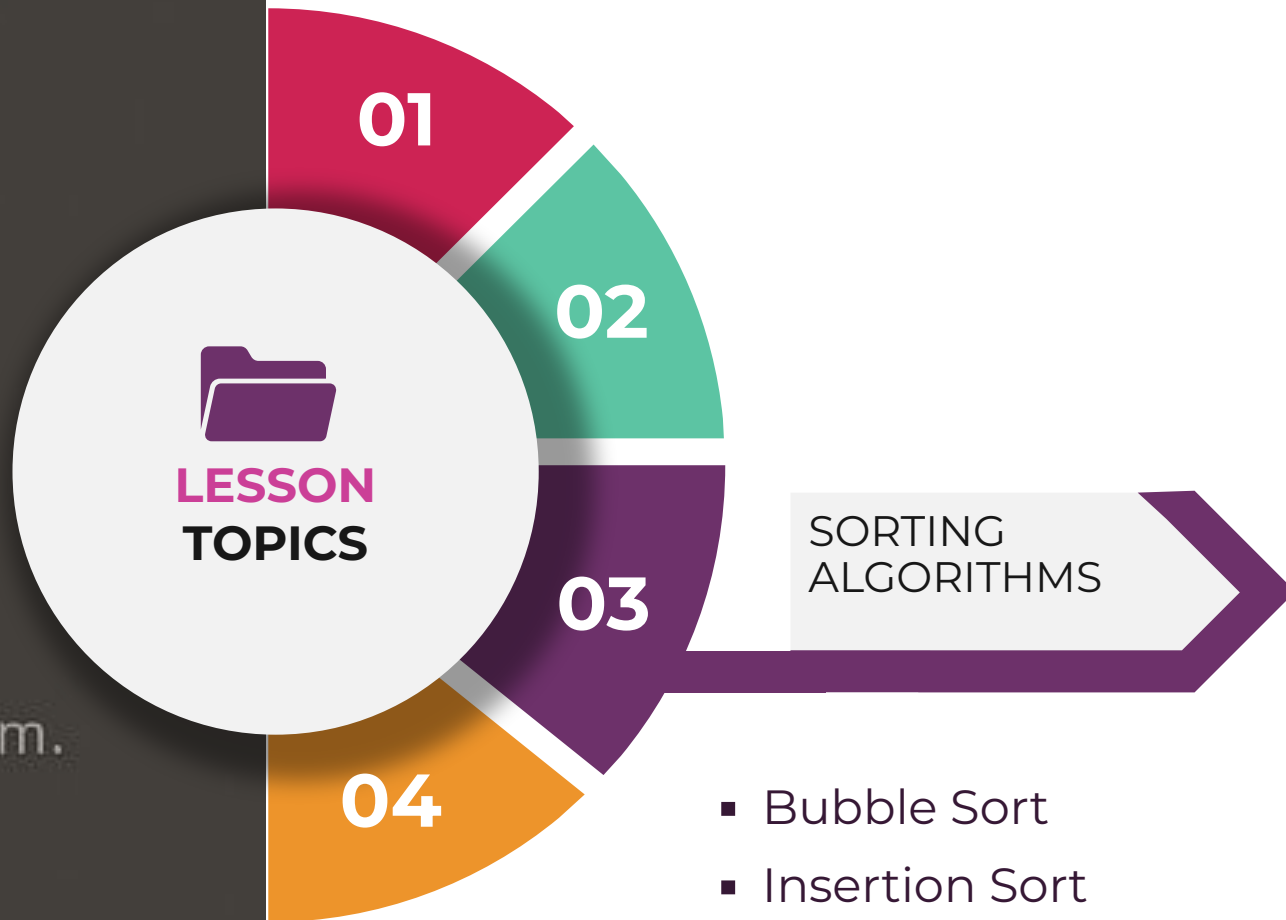
Problem Solving
Process

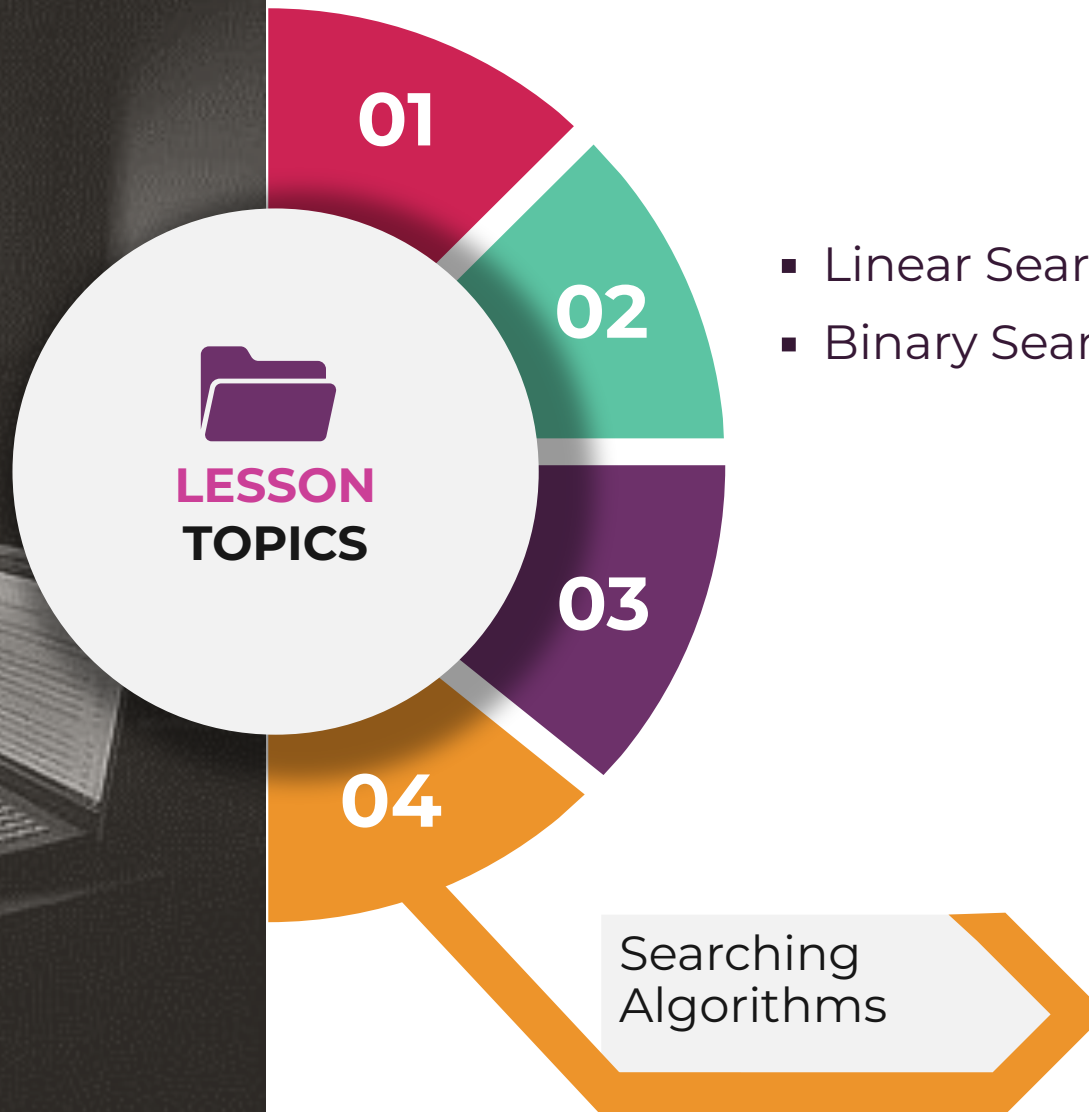






Programming is an art form.







Linear Search

is used on a collections of items. It relies on the technique of traversing a list from start to end by exploring properties of all the elements that are found on the way.



Linear Search

- In this type of search, a sequential search is made over all items one by one.
- Every item is checked and if a match is found then that particular item is returned, otherwise the search continues till the end of the data collection.

Linear Search



Linear Search

- Start from the first element, compare k with each element x .

2	4	0	1	9
---	---	---	---	---

Array to be searched for

$k = 1$

2	4	0	1	9
---	---	---	---	---

↑
 $k \neq 2$

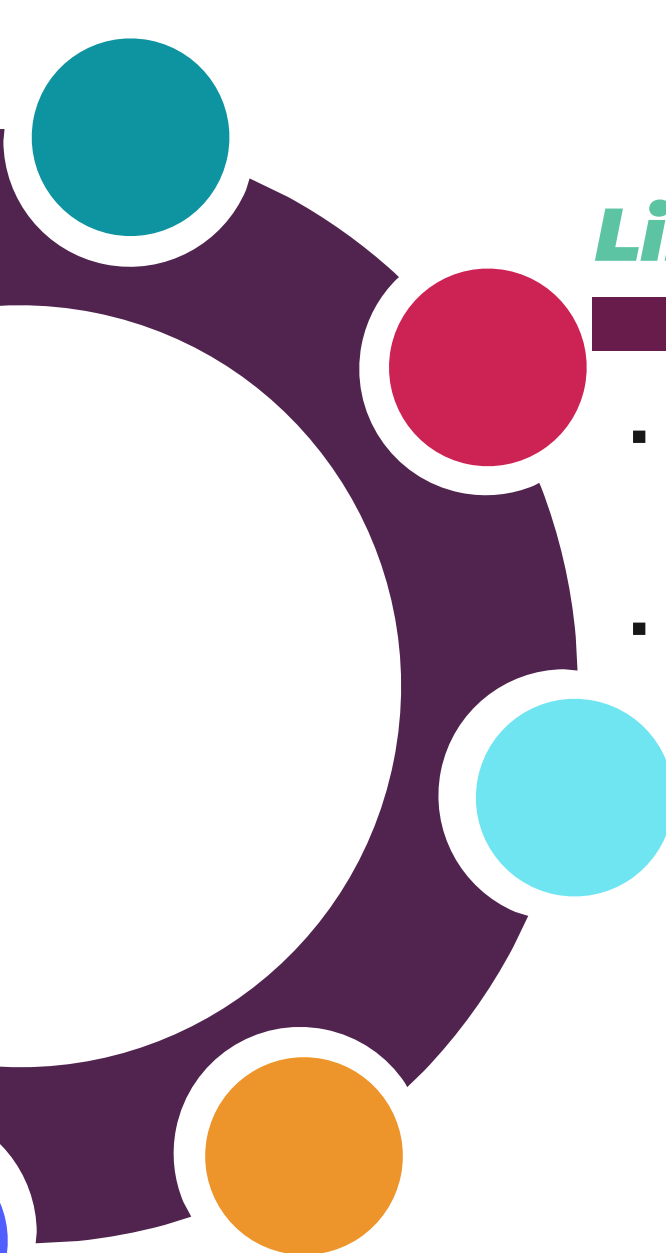
2	4	0	1	9
---	---	---	---	---

↑
 $k \neq 4$

2	4	0	1	9
---	---	---	---	---

↑
 $k \neq 0$

Compare with each element



Linear Search

- If $x == k$, return the index.
- Else, return not found.

2	4	0	1	9
---	---	---	---	---

Array to be searched for

2	4	0	1	9
---	---	---	---	---

↑
k = 1

Element found

Linear Search

Linear Search for "2" in 6 elements array

Given Array

5	9	10	2	90	4
0	1	2	3	4	5

5 != 2

5	9	10	2	90	4
0	1	2	3	4	5

9 != 2

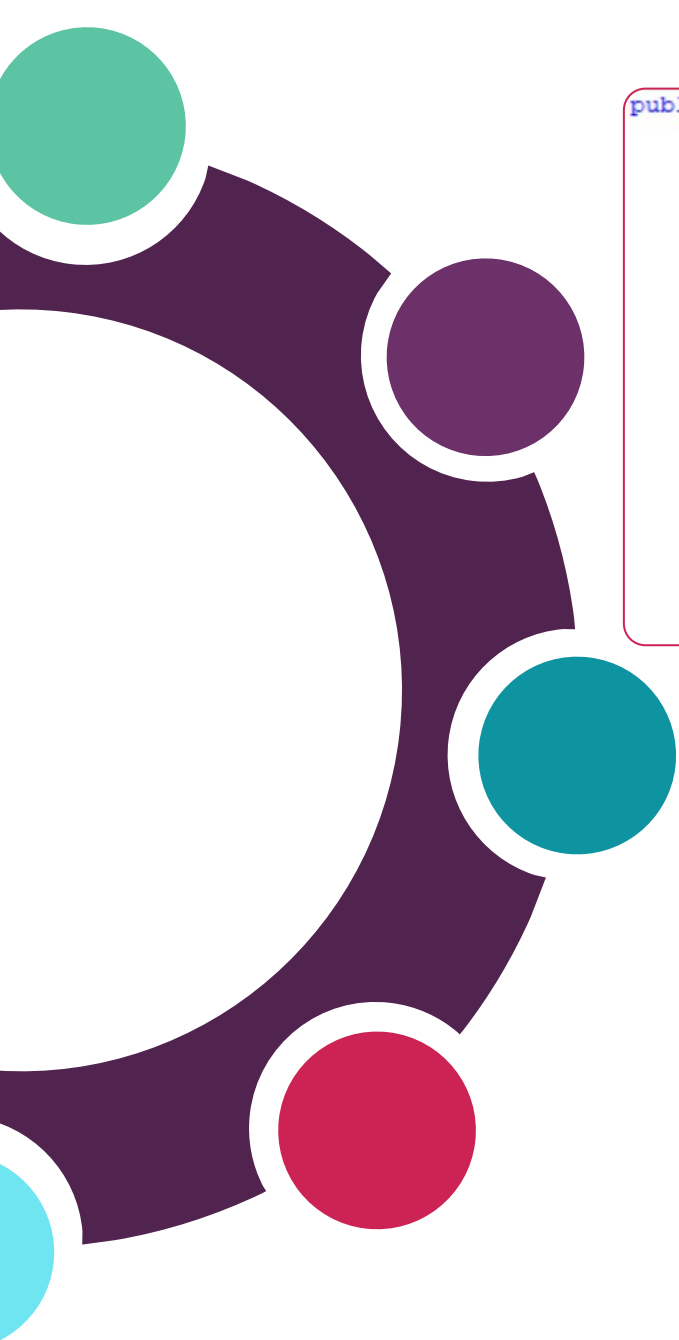
5	9	10	2	90	4
0	1	2	3	4	5

10 != 2

5	9	10	2	90	4
0	1	2	3	4	5

2 found at index 3.





```
public class LinearSearch1 {
    public static int linearSearch(int[] inputArray,int key){
        for(int i=0;i<inputArray.length;i++){
            if(inputArray[i] == key){
                return i;}
        }
        return -1;
    }

    public static void main(String args[]){
        int[] arr1 = {5,9,10,2,90,4};
        int key = 2;
        int result = linearSearch(arr1,key);
        System.out.println( (result != -1) ? "Required key: "+key+" found at index:- "+result : "Key "+key+" not found");
        key = 18;
        result = linearSearch(arr1,key);
        System.out.println( (result != -1) ? "Required key:- "+key+" found at index:- "+result : "Key "+key+" not found");
    }
}
```

Sample Program

Linear Search





Linear Search

Sample Program

```
int[] arr1 = {5,9,10,2,90,4};  
int key = 2;  
int result = linearSearch(arr1,key);
```

```
public static int linearSearch(int[] inputArray,int key){  
    for(int i=0;i<inputArray.length;i++){  
        if(inputArray[i] == key){  
            return i; }  
        }  
    return -1;
```

```
int result = linearSearch(arr1,key);  
System.out.println( (result != -1) ? "Required key: "+key+"  
    found at index:- "+result : "Key "+key+ " not found");
```



Linear Search

Sample Program

```
int[] arr1 = {5,9,10,2,90,4};  
key = 18;  
result = linearSearch(arr1,key);
```

```
public static int linearSearch(int[] inputArray,int key){  
    for(int i=0;i<inputArray.length;i++){  
        if(inputArray[i] == key){  
            return i; }  
    }  
    return -1;
```

```
int result = linearSearch(arr1,key);  
System.out.println( (result != -1) ? "Required key: "+key+"  
    found at index:- "+result : "Key "+key+ " not found");
```



Binary Search

A binary search locates an item in a sorted array by repeatedly dividing the search interval in half.

In binary search, we first compare the key with the item in the middle position of the array.

- If there's a match, we can return immediately. If the key is less than the middle key, then the item sought must lie in the lower half of the array; if it's greater then the item sought must lie in the upper half of the array.



Binary Search

- So we repeat the procedure on the lower (or upper) half of the array
- Finding the middle is often coded as

$$\text{mid} = (\text{high} + \text{low})/2;$$

- This overflows if high and low are close to the largest expressible integer. The following gives the same result and never overflows, if high and low are non-negative.

$$\text{mid} = \text{low} + (\text{high} - \text{low})/2;$$

Binary Search

- Begin with the mid element of the whole array as a search key.
- If the value of the search key is equal to the item then return an index of the search key.
- Or if the value of the search key is less than the item in the middle of the interval, narrow the interval to the lower half.
- Otherwise, narrow it to the upper half.
- Repeatedly check from the second point until the value is found or the interval is empty.

Binary Search for 50 in 7 elements Array

Given Array

1	5	20	35	50	65	70
---	---	----	----	----	----	----

0 1 2 3 4 5 6
start end

$$\text{mid} = \frac{0+6}{2} = 3$$

1	5	20	35	50	65	70
---	---	----	----	----	----	----

0 1 2 3 4 5 6
start end

$$\text{mid} = \frac{4+6}{2} = 5$$

1	5	20	35	50	65	70
---	---	----	----	----	----	----

0 1 2 3 4 5 6
start end

$$\text{mid} = \frac{4+4}{2} = 4$$

1	5	20	35	50	65	70
---	---	----	----	----	----	----

0 1 2 3 4 5 6
start end

35 < 50
Take 2nd Half

65 > 50
Take 1st Half

50 Found
Return 50



Binary Search

- Begin with the mid element of the whole array as a search key.
- If the value of the search key is equal to the item then return an index of the search key.
- Or if the value of the search key is less than the item in the middle of the interval, narrow the interval to the lower half.
- Otherwise, narrow it to the upper half.
- Repeatedly check from the second point until the value is found or the interval is empty.

Binary Search

	0	1	2	3	4	5	6	7	8	9
Search 23	2	5	8	12	16	23	38	56	72	91
	L=0	1	2	3	M=4	5	6	7	8	H=9
23 > 16 take 2 nd half	2	5	8	12	16	23	38	56	72	91
	0	1	2	3	4	L=5	6	M=7	8	H=9
23 < 56 take 1 st half	2	5	8	12	16	23	38	56	72	91
	0	1	2	3	4	L=5, M=5	H=6	7	8	9
Found 23, Return 5	2	5	8	12	16	23	38	56	72	91



Binary Search

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

search element **12**

Step 1:

search element (12) is compared with middle element (50)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

12

Both are not matching. And 12 is smaller than 50. So we search only in the left sublist (i.e. 10, 12, 20 & 32).

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

Step 2:

search element (12) is compared with middle element (12)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

12

Both are matching. So the result is "Element found at index 1"

Binary Search

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

Search element 80

Step 1:

search element (80) is compared with middle element (50)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

80

Both are not matching. And 80 is larger than 50. So we search only in the right sublist (i.e. 55, 65, 80 & 99).

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

Step 2:

search element (80) is compared with middle element (65)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

80

Both are not matching. And 80 is larger than 65. So we search only in the right sublist (i.e. 80 & 99).

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

Step 3:

search element (80) is compared with middle element (80)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

80

Both are not matching. So the result is "Element found at index 7"



Binary Search

```
public class BinarySearch1 {
    public static int binarySearch(int[] inputArray,int key) {
        int start = 0;
        int end = inputArray.length-1;
        while (start <= end) {
            int mid = (start + end) / 2;
            if (key == inputArray[mid]) {
                return mid; }
            if (key < inputArray[mid]) {
                end = mid - 1;}
            else {
                start = mid + 1; }
        }
        return -1; }
}
```

```
public static void main(String args[]){
    int[] arr1 = {1,5,20,35,50,65,70};
    int key = 100;
    int result = binarySearch(arr1,key);
    System.out.println( (result != -1) ? "Required key: "+key+" found at index:- "+result : "Key "+key+" not found");
    key = 50;
    result = binarySearch(arr1,key);
    System.out.println( (result != -1) ? "Required key: "+key+" found at index: "+result : "Key "+key+" not found");
}
```



Binary Search

```
int[] arr1 = {1,5,20,35,50,65,70};
```

```
key = 100
```

```
int start = 0;
int end = inputArray.length - 1;
while (start <= end) {
    int mid = (start + end) / 2;
    if (key == inputArray[mid]) {
        return mid;
    }
    if (key < inputArray[mid]) {
        end = mid - 1;
    } else {
        start = mid + 1;
    }
}
return -1;
}
```




Binary Search

```
int[] arr1 = {1,5,20,35,50,65,70};
```

```
key = 50
```

```
int start = 0;
int end = inputArray.length - 1;
while (start <= end) {
    int mid = (start + end) / 2;
    if (key == inputArray[mid]) {
        return mid;
    }
    if (key < inputArray[mid]) {
        end = mid - 1;
    } else {
        start = mid + 1;
    }
}
return -1;
}
```



Thanks!
Questions?